Loup M. Marquez Luttringer – Project Manager

Matheus Lopez – Back-End & Front-End Developer

Noah Gibelli – AI Developer

Version: v1.0

Date: October 30, 2025

Horus Project – Observability Implementation Justification

(Valorize Observability, adopting technologies to ensure observability, Hybrid Logging and Metrics & Our testing strategy)

Point 1 – A – Valorize Observability

When you develop a program like Horus, in which every part of the code is divided into several small components, understanding what happens in real time becomes a vital part of the architecture. Each agents have to execute different tasks gave by clients. When a client can do everything that he wants you have to be careful with exception redirection for example. Without proper observability, it becomes impossible to monitor the system's health, detect errors, or analyze performance.

That's why we decided to make observability a core principle of our infrastructure, not an optional tool.
It helps us:

- Detect issues faster.
- Understand system behavior.
- Guarantee reliability and stability for future deployments.

Observability is, in short, the eyes and ears of Horus.

A multi-agent system like Horus is highly asynchronous — agents exchange messages, tasks, and results through RabbitMQ. Without visibility, errors can propagate silently from one agent to another. Observability is the foundation of reliability. Thanks to continuous logging, metrics, and testing, Horus can not only detect problems but also anticipate them. It gives developers and users confidence that the platform remains stable, scalable, and ready for real deployment.

For example:

- An agent can stop sending heartbeats.
- A message can be delayed or lost in the queue.
- A container can crash without the orchestrator knowing it.

To solve these problems, we implemented a hybrid observability layer combining metrics, logs, and testing pipelines.

This approach allows us to:

- Track agent health in real time.
- Log up every event and categorize it (SYSTEM, AGENT, SECURITY, USER).

By adopting this proactive monitoring mindset, we ensure to maintain the health and safety of our program.

Point 1 – B – Adopting technologies to ensure observability

Component	Technology	Role
Logger	Python logging + custom hybrid logger	Records all system events, categorized and timestamped
Database Logs	PostgreSQL (CriticalLogs table)	Persistent audit trail for compliance and debugging
Message Monitoring	RabbitMQ management interface	Visualizes message flow and agent activity
Testing	pytest, coverage.py	Ensures quality and code integrity

All these technologies work together to provide complete transparency and reliability. They make Horus **predictable, measurable, and auditable** — three essential qualities of enterprise software.

Point 1 – C – Hybrid Logging and Metrics

We developed a hybrid logging system to merge console visibility and database persistence. Each log includes:

- Timestamp
- Component name
- Log level (INFO, WARNING, ERROR)
- Message details

Example:
[SECURITY] [2025-10-29 12:00:41] - Admin user logged in successfully [AGENT] [2025-10-29 12:00:45] - Agent “retriever_01” sent heartbeat

Logs are not only displayed live — they are also stored in PostgreSQL for later analysis. This system allows developers to review historical data and understand what happened during production incidents.

With this, every event can be traceable, we reduce drastically the risk of unsolved problems, and solving problems become simpler and cleaner because we don’t have to overwrite the code to put so print debugs. Inside the code, each function has at least one line of logs to resume the information that transit and transformations that have been made inside the function .

Point 1 – D – Our testing strategy

Testing is the foundation of software reliability. Our project's philosophy is that each release, no matter what it contains, has to be tested as much as possible. Each line of code has to be tested at the moment that the code is modified. We implemented several types of automated tests:

- Unit Tests: Validate individual modules such as authentication, task routing, and agent registry. But we never test routes themselves because they require having all of the infrastructure on. Role of these tests are to test single functions.
- Integration Tests: Ensure orchestrator ↔ agent ↔ queue communication works correctly.
- Coverage Analysis: Measures how much of the code is tested (goal: above 80% on each python file).
- Functional Tests: Validate combinations of sequential API calls. At this moment, we are trying to reproduce the experience of a simple user that wants to test every route.

This guarantees that Horus can evolve without losing stability. Each one of these types of tests is a reproduction of our mentality. With tests results, we plan what to do next.